HANNIBAL V2

Contract Data Pipeline and Fine-Tuning Readiness Report

Organization	E-Tafakna
System	Hannibal V2 — AI Legal Contract Reasoning Engine
Repository scope	Source-grounded contract ingestion, runtime preparation, Agent 1/Agent 2 data handoff, and adapter preparation
Report date	17 August 2026

Core doctrine: legal content comes exclusively from documentary indexes retrieved through RAG. Fine-tuning improves reasoning and drafting; it is not a source of law.

Table of Contents

Executive Summary 6

1. Project Context and Problem 7

1.1 Product definition . . . 7
1.2 Problems identified by the R&D; specification . . . 7
1.3 Engineering objective . . . 7
1.4 Definition of success for this repository . . . 8

2. Source Basis and Governing Constraints 8

2.1 Sources of truth . . . 8
2.2 Non-negotiable system rules . . . 8
2.3 Required runtime sequence . . . 9
2.4 Fine-tuning boundary . . . 9

3. Hannibal V2 Architecture 10

3.1 Agent 1 — Information Collector . . . 10
3.2 Agent 2 — Hannibal Core . . . 10
3.3 Agent 3 — Document Formatter . . . 10
3.4 Documentary indexes . . . 10

4. Repository Structure and Implemented Components 11

4.1 Top-level organization . . . 11
4.2 Principal modules . . . 12
4.3 Design principles visible in the code . . . 12

5. End-to-End Data Pipeline 13

5.1 Stage 0 — Environment and verification . . . 13
5.2 Stage 1 — Inventory . . . 13
5.3 Stage 2 — Format-aware extraction . . . 14
5.4 Stage 3 — Normalization and stable blocks . . . 14
5.5 Stage 4 — Labeling, assembly, and validation . . . 14
5.6 Stage 5 — Fixed pilot . . . 14
5.7 Stage 6 — Full corpus . . . 14

6. Canonical Contract Templates and Review 15

6.1 Canonical record purpose . . . 15
6.2 Classification and template state . . . 15
6.3 Review workspace . . . 15
6.4 Review dimensions . . . 15
6.5 Eligibility safeguard . . . 16

7. Qwen Enrichment and Compact Runtime Records 16

7.1 Why enrichment is separate . . . 16
7.2 Enrichment commands . . . 16
7.3 Compact runtime projection . . . 17
7.4 Runtime record role . . . 17

8. Agent 1 Demonstration Inputs 17

8.1 Fixed interface . . . 17
8.2 Generation modes . . . 18
8.3 Command . . . 18
8.4 Measured outcome . . . 18
8.5 Sanitization before Agent 2 . . . 19

9. Agent 2 Grounding and Target Reconstruction 19

9.1 Generation-facts record . . . 19
9.2 Retrieved evidence package . . . 19
9.3 Target reconstruction . . . 19
9.4 Contract-quality checks . . . 19

10. Alpaca Dataset Export 20

10.1 Standard row . . . 20
10.2 Export command . . . 20
10.3 Per-record outputs . . . 20
10.4 Current export result . . . 21
10.5 Production mode . . . 21

11. Fine-Tuning Implementation and Handoff 21

11.1 Scope inside Agent 2 . . . 21
11.2 Implemented QLoRA profile . . . 22
11.3 Grouped split and leakage control . . . 22
11.4 Dataset audit before model loading . . . 22
11.5 Source profile versus implemented profile . . . 23

12. Fine-Tuning and Packaging Commands 23

12.1 Install the isolated stack . . . 23
12.2 Exact-token preflight . . . 23
12.3 Execute the guarded demonstration profile . . . 23
12.4 Resume from a checkpoint . . . 23
12.5 Override profile values explicitly . . . 24
12.6 Merge and register with Ollama . . . 24

13. Verified Results 24

13.1 Pilot results . . . 24
13.2 Full-corpus results . . . 25
13.3 Downstream results . . . 25
13.4 Automated test baseline . . . 25
13.5 Interpretation . . . 26

14. Quality, Safety, and Audit Controls 26

14.1 Source integrity . . . 26
14.2 Grounding and hallucination controls . . . 26
14.3 Privacy controls . . . 26
14.4 Placeholder and completeness controls . . . 26
14.5 Status and eligibility controls . . . 26
14.6 KPI discipline . . . 27

15. Critical Ambiguities and Decision Gates 27

15.1 Architecture and dataset ownership . . . 27
15.2 Required-section KPI . . . 27
15.3 Schedule . . . 27
15.4 Runtime payloads . . . 27
15.5 Adapter routing . . . 28
15.6 Evaluation . . . 28
15.7 Non-contract destination . . . 28

16. Deployment-Readiness Gates and Recommendations 28

16.1 Data readiness . . . 28
16.2 Infrastructure readiness . . . 28
16.3 Evaluation readiness . . . 29
16.4 Runtime integration readiness . . . 29
16.5 Recommended execution order . . . 29

17. Conclusion 29

Appendix A. Complete Command Reference 30

A.1 Base environment and tests . . . 30
A.2 Inventory and extraction . . . 30
A.3 Pilot and corpus . . . 30
A.4 Review and enrichment . . . 30
A.5 Runtime, Agent 1, and Alpaca . . . 30
A.6 Fine-tuning handoff and packaging . . . 30
A.7 Rebuild this report . . . 31

Appendix B. Schema and Artifact Matrix 31

Appendix C. Artifact Directory Guide 32

Appendix D. Glossary 33

Appendix E. Source Traceability 34

E.1 Report integrity statement 34

E.2 Referenced repository documents 34

Executive Summary

Hannibal V2 is E-Tafakna's AI Legal Contract Reasoning Engine. It is designed to produce professional contracts adapted to multiple MENA jurisdictions through a controlled combination of structured information collection, documentary retrieval, legal reasoning, validation, risk analysis, and document formatting. The governing doctrine is strict: **the model must never invent law**. Legal content comes exclusively from documentary indexes retrieved through RAG; model adaptation is limited to reasoning, organization, drafting, reformulation, legal style, clause choice, and global coherence.

This repository establishes the engineering path required to transform a heterogeneous collection of contract files into traceable, schema-validated records suitable for runtime retrieval and an Agent 2 fine-tuning handoff. The implemented path begins with PDF, DOC, and DOCX documents under `templates/`. It inventories and hashes every physical file, detects exact duplicates, extracts ordered text without modifying the legal sources, normalizes that text, assembles evidence-bearing canonical JSON, validates the result, and routes uncertain or sensitive material through explicit review controls. A separate enrichment stage can use a local Qwen model for coordinate-bound classification while deterministic code remains responsible for schema validation and final assembly.

The pipeline then projects eligible or explicitly demonstrative canonical records into compact runtime templates. It creates Agent 1 information-collector payloads, removes forbidden session and user identifiers from the model-facing Agent 2 view, attaches the exact retrieved documentary evidence, reconstructs a grounded target contract from source order, and exports strict Alpaca rows containing only `instruction`, `input`, and `output`. Every exported row is paired with a non-model manifest that preserves source, template, clause, path, hash, language, jurisdiction, review, and eligibility information.

The measured repository state provides a concrete engineering result. The corpus contains **1,276 physical source files** representing **1,275 unique content groups**. Full-corpus processing recorded **1,273 successful extractions** and two encrypted groups, produced **660 contract-like demonstration candidates**, and excluded 615 groups from that route. The enrichment and compact-runtime path produced **90 runtime demonstration records**. Agent 1 preparation produced **65 valid records**, and the grounded Agent 2 export produced **65 Alpaca rows** while recording 25 rejected runtime records. The exported language distribution is 36 English, 26 Arabic, and three French rows. Source integrity checks report that all original template hashes remained unchanged. The repository's current automated baseline is **51 tests passed in 55.40 seconds** on 17 August 2026.

These outputs are deliberately described as pipeline and readiness results. The current exported dataset is marked `demo_unreviewed`, its rows are marked `training_eligible: false`, and its volume is below the source-defined minimum of 200 eligible examples. This prevents engineering artifacts from being misrepresented as lawyer-approved legal data. The repository nevertheless includes a guarded QLoRA entry point that audits dataset hashes and eligibility, tokenizes without truncation, groups related sources into deterministic 80/10/10 splits, supervises assistant completion tokens only, trains Agent 2 LoRA parameters, and can merge and package a final model for Ollama.

The result is a reproducible, source-grounded foundation for the next controlled stage: review, dataset expansion, adapter execution on suitable GPU infrastructure, measured evaluation, and integration into Agent 2 without weakening the RAG-only legal-content boundary.

> **Report scope.** This report records implemented behavior and measurements found in the repository. It does not claim that legal correctness, production KPIs, lawyer acceptance, or deployment readiness has been established.

1. Project Context and Problem

1.1 Product definition

The project specification defines Hannibal as a Legal Contract Reasoning Engine rather than a chatbot or a simple GPT contract generator. Its expected output is not merely plausible text. A useful result must combine a coherent legal structure, clauses adapted to the user's context, conformity with the relevant jurisdiction through documentary evidence, and drafting quality close to professional legal work.

This distinction determines the repository design. A conventional text-generation demonstration might send a prompt directly to a language model and save its response. Hannibal instead requires a chain of controlled transformations. User facts must be separated from legal sources; the relevant indexes must be selected; documentary evidence must be retrieved; clauses must be selected and organized; the result must be validated for omissions, placeholders, contradictions, and unsupported legal statements; risks must be analyzed; and only then may a formatter produce the final document.

1.2 Problems identified by the R&D specification

The supplied R&D source identifies recurring weaknesses that motivate model adaptation and better data engineering:

1. Clauses may be vague or generic instead of being adapted to the contract context.
1. Contract structure may be inconsistent across generated documents.
1. Essential clauses may be omitted.
1. Drafting may be repetitive, unnecessarily long, or recognizably “LLM-like.”
1. MENA cultural and drafting adaptation may be limited.
1. Dates and monetary amounts may use incorrect formats.

These problems are connected. Incomplete or poorly structured source data makes it harder to select clauses, preserve jurisdiction, replace variables, or measure completeness. A fine-tuning file assembled without evidence and review controls can also teach formatting patterns while obscuring the origin of its legal content. The repository therefore treats dataset construction as an auditable system rather than a one-time conversion script.

1.3 Engineering objective

The implementation objective is to create a deterministic and reviewable route from raw legal documents to Agent 2 examples while preserving the source-defined boundaries. The route must:

1. Read the complete heterogeneous corpus without altering it.
2. Preserve source paths and cryptographic hashes.
3. Extract text in document order and record extraction failures rather than guess.
4. Separate deterministic evidence from model-assisted labels.
5. Validate every schema and every coordinate-grounded transformation.
6. Keep Agent 1 limited to facts and Agent 3 limited to formatting.
7. Ensure Agent 2 receives documentary evidence before legal text is used.
8. Keep Alpaca's `instruction`, `input`, and `output` fields semantically distinct.
9. Prevent demonstration records from silently becoming production training data.
10. Provide a guarded, reproducible fine-tuning handoff.

1.4 Definition of success for this repository

Success at the repository level is demonstrated through reproducibility, artifact integrity, traceability, validation, and truthful status. It is not inferred from the existence of JSON files alone. The report therefore uses the following evidence:

Evidence category	Repository evidence	Meaning
Corpus coverage	Inventory and full-corpus summaries	Every supported physical file is accounted for
Source integrity	Before/after SHA-256 comparison	Legal source files were not changed
Structural correctness	JSON Schema and semantic validation	Artifacts satisfy explicit data contracts
Grounding	Coordinates, source IDs, clause IDs, hashes	Generated targets can be traced to documentary text
Boundary enforcement	Agent 1, Agent 2, and runtime validators	User identifiers and legal responsibilities remain separated
Dataset truthfulness	Export manifest and summary statuses	Demonstration rows cannot be confused with reviewed production rows
Regression safety	Automated test suite	Deterministic and safety behavior remains executable

2. Source Basis and Governing Constraints

2.1 Sources of truth

All material legal, product, architectural, training, and operational statements in this report are derived from the repository's approved sources:

1. `Hannibal V2.pdf`, sections 1–6: purpose, operating principles, three-agent architecture, runtime flow, and fine-tuning boundary.
1. `Hannibal R&D.pdf`, sections 7–14: observed problems, objectives, datasets, fine-tuning strategy, infrastructure, roadmap, and KPIs.
1. `contrat_example.md`: example blueprint, contract-and-question, risk-clause, and legal-clause record families.
1. Repository plans, schemas, code, tests, and generated summaries: implemented behavior and measured outputs.

No external legal rule, clause, jurisdictional requirement, vector value, or performance result is introduced. Where the supplied sources disagree or omit a decision, this report preserves the alternatives as `AMBIGUITY`.

2.2 Non-negotiable system rules

The following rules govern every pipeline stage:

1. Hannibal is a Legal Contract Reasoning Engine, not a conversational assistant.
1. Agent 1 is an Information Collector. It gathers facts and never drafts a contract.
1. Agent 2 is Hannibal Core. It owns request analysis, index choice, RAG retrieval, clause selection, construction, validation, and risk analysis.
1. Agent 3 is a Document Formatter. It owns layout and DOCX/PDF/HTML export and contains no legal logic.
1. The model must never invent law. Legal content comes from retrieved documentary indexes.
1. Fine-tuning improves reasoning and drafting behavior; it does not replace legal indexes or teach the model the law.
1. Jurisdiction, language, source tracking, and audit metadata must be preserved when available.
1. A contradiction in the specification must be reported, not silently repaired.

2.3 Required runtime sequence

The architecture fixes the operating order:

1. Collect user information.
2. Analyze the requested contract.
3. Choose the relevant indexes.
4. Retrieve documentary evidence through RAG.
5. Select clauses.
6. Generate the contract.
7. Validate it.
8. Analyze risks.
9. Produce the formatted final document.

This sequence is also a data-governance rule. Fine-tuning cannot be allowed to move legal reasoning into Agent 1, and Agent 3 cannot decide which clause or legal rule applies. Agent 2 may load specialized adapters dynamically, but those adapters operate inside the legal-reasoning stage and remain downstream of retrieval.

2.4 Fine-tuning boundary

The source material is explicit about what adaptation may improve: structure, clause choice, organization, drafting, reformulation, legal style, MENA adaptation, and global coherence. It is equally explicit about what adaptation may not become: a source of law. This yields two distinct data channels:

Channel	Carries	Authority
Agent/user facts	Contract type, country, language, parties, amounts, duration, specific facts	User and Agent 1 payload
Documentary evidence	Blueprint structure, retrieved clauses, risk evidence, source metadata	RAG indexes and reviewed source records

The training example may teach Agent 2 how to transform these inputs into a well-structured target. It may not hide unsupported law inside the target or rely on the base model to supply missing legal content.

> **Safety invariant.** A well-written clause is not acceptable merely because it sounds legal. Its legal content must remain attributable to retrieved documentary evidence.

3. Hannibal V2 Architecture

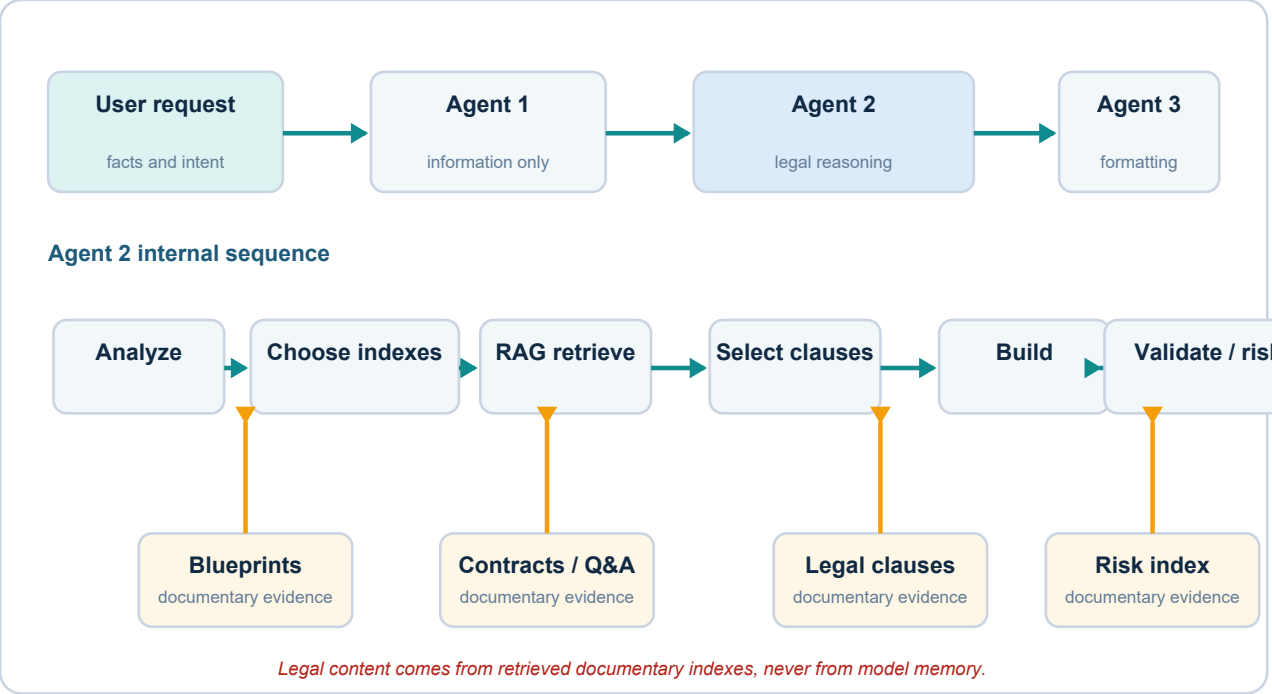


Figure 1. Hannibal V2 three-agent architecture and the law-from-RAG boundary.

3.1 Agent 1 — Information Collector

Agent 1 gathers the contract type, country, language, parties, amounts, duration, and additional context required before generation. Its fixed payload includes metadata, contract type and subtype, party records, dynamic principal information, general information, missing-information markers, and other facts. The raw payload also contains `session_id` and `user_id` for application traceability.

The repository preserves this raw payload as a non-model audit artifact. The Agent 2 intake adapter creates a separate model-facing projection and removes `session_id` and `user_id`. Missing required facts remain explicit, including literal `AMBIGUITY` values when the source data cannot resolve them. Agent 1 does not create legal clauses, select law, or draft the contract.

3.2 Agent 2 — Hannibal Core

Agent 2 is the only component that owns legal reasoning. Its required internal sequence is visible in the architecture diagram: analyze the request, choose indexes, retrieve evidence, select clauses, construct the document, validate coherence, and analyze risk. The current repository focuses on preparing the data and controls required for this stage.

The model-facing Agent 2 record includes document facts, parties, variable bindings, unmapped facts, and warnings. A separate retrieval package supplies the template and clause evidence. A target builder reconstructs the contract from ordered extraction blocks and records passage-level grounding. The final Alpaca row serializes the task and evidence without including application identifiers or audit-only manifest fields.

3.3 Agent 3 — Document Formatter

Agent 3 receives the generated and validated contract. It handles layout, numbering, tables of contents, signatures, and final export to DOCX, PDF, or HTML. The current repository does not move legal selection, validation, or risk analysis into this formatting stage. Report generation itself is separate from the Hannibal runtime and does not represent Agent 3 implementation.

3.4 Documentary indexes

The example source describes several record families rather than a single ready-to-train JSON document:

1. Blueprint/template metadata with required and conditional sections, variable points, required elements, adaptation hints, retrieval preferences, and an example vector field.
1. Tunisian contract context with repeated questions, answers, offsets, answer-presence flags, and a vector field.
1. Risk-clause metadata with detection patterns, severity, legal basis, recommendation, impact, probability, keywords, embedding text, and a vector field.
1. Legal-clause text with an identifier and vector field.

The displayed `...` fragments are not complete embeddings and are never treated as usable vectors. The repository's canonical records are provider-neutral and contain no invented embeddings or vector-database configuration.

4. Repository Structure and Implemented Components

4.1 Top-level organization

The repository separates legal sources, schemas, configuration, executable pipeline code, review state, generated artifacts, prompts, feature decisions, and tests.

Area	Purpose
`templates/`	Immutable PDF, DOC, and DOCX source corpus
`schemas/`	Strict JSON Schemas for every material handoff
`config/`	Controlled taxonomy and fixed pilot manifest
`prompts/`	Coordinate-only model instructions
`src/`	Inventory, extraction, normalization, assembly, runtime, export, and fine-tuning code
`review_ui/` and `review_app.py`	Local server-rendered review dashboard
`review-workspace/`	Resumable working copies and review audit state
`artifacts/`	Generated manifests, records, summaries, and dataset exports
`features/`	Approved feature-level decisions and interface definitions
`tests/`	Automated safety, schema, and pipeline coverage

4.2 Principal modules

Module	Implemented responsibility
`src/inventory.py`	Corpus enumeration, media classification, SHA-256 hashing, count enforcement, and exact-duplicate groups
`src/extract.py`	PDF, DOCX, and legacy DOC extraction with read-only boundaries
`src/normalize.py`	Unicode NFC, line endings, mappings, stable IDs, language hints, and placeholder normalization
`src/ollama_extract.py`	Sequential structured labeling through local Ollama with schema and coordinate checks
`src/assemble.py`	Deterministic canonical assembly and unparsed-content preservation
`src/validate.py`	Schema, hash, coordinate, placeholder, order, eligibility, and provider-neutrality validation
`src/pilot.py`	Fixed 36-file pilot orchestration and before/after source verification
`src/full_corpus.py`	Resumable processing of every supported source and duplicate-aware routing
`src/qwen_enrichment.py`	Contract-only packet creation, local Qwen execution, and separate merge output
`src/runtime_templates.py`	Compact runtime record projection with production and demo gates
`src/agent1_demo_inputs.py`	Grounded demonstration payload proposals and deterministic Agent 1 assembly
`src/agent2_intake.py`	Sanitized model-facing generation facts and validation provenance
`src/contract_target_builder.py`	Evidence package and ordered contract-target reconstruction
`src/alpaca_export.py`	Strict Alpaca rows plus source-rich manifests and rejection records
`src/finetune_slm.py`	Guarded QLoRA audit, tokenization, split, training, evaluation, merge, and Ollama packaging

4.3 Design principles visible in the code

The implementation consistently follows several engineering patterns:

1. **Immutability of source documents:** extraction reads source files and all generated material is written under artifact or review directories.
1. **Stable identity:** hashes and deterministic IDs connect physical files, content groups, canonical templates, runtime records, clauses, and Alpaca rows.
1. **Provider neutrality:** the canonical record is independent of a particular vector database or model provider.
1. **Determinism around model calls:** the local model may propose labels, but schemas, coordinates, assembly, eligibility, and status remain code-controlled.
1. **Explicit rejection:** encrypted, malformed, empty, unsupported, ungrounded, or ineligible records are preserved with reasons instead of being silently dropped or repaired.
1. **Resumability:** expensive full-corpus and model-assisted stages reuse compatible completed work unless an explicit force option is supplied.

5. End-to-End Data Pipeline



Figure 2. Repository data path from source contracts to the Agent 2 adapter handoff.

The data path uses a canonical evidence layer between source extraction and every downstream representation. This prevents the compact runtime record or the Alpaca row from becoming the only copy of the source relationship. Each transformation narrows the information for a specific consumer while keeping an audit link to the richer upstream artifact.

5.1 Stage 0 — Environment and verification

The base pipeline uses Python 3.12 and dependencies from `requirements.txt`. The fine-tuning stack is isolated in `requirements-finetune.txt` because its PyTorch build must match the CUDA environment.

```
python -m pip install -r requirements.txt
python -m pytest -q
```

The test command is both an installation check and a regression gate. Model and corpus commands should be run from the repository root so their relative artifact paths resolve consistently.

5.2 Stage 1 — Inventory

Inventory recursively identifies supported sources, calculates SHA-256 hashes, records size, extension, media type, and folder hints, and groups exact duplicate content without erasing physical paths.

```
python src\inventory.py --templates-dir templates --output artifacts\manifest.jsonl --strict-counts
```

`--strict-counts` enforces the recorded corpus shape. The current baseline is 955 PDF files, 297 DOC files, and 24 DOCX files, for 1,276 physical files total. One exact duplicate group results in 1,275 unique content groups.

5.3 Stage 2 — Format-aware extraction

A single file can be inspected through the extraction command:

```
python src\extract.py templates\Tunisie\contratdetravail.pdf --output artifacts\single-extraction.json
```

An optional manifest record can bind the extraction to a precomputed inventory identity:

```
python src\extract.py <source-path> --output <extraction.json> --manifest-record <manifest-record.json>
```

PDF extraction is page based. DOCX extraction reads OOXML paragraphs, tables, headers, and footers. On Windows, legacy DOC extraction uses Microsoft Word in a separate hidden process with alerts, macros, link updates, recent-file registration, and saving disabled. A non-Windows fallback boundary can invoke LibreOffice in headless safe mode and write only to a temporary directory. No OCR implementation is present; likely scanned, unreadable, encrypted, or empty records are identified and routed rather than guessed.

5.4 Stage 3 — Normalization and stable blocks

Normalization applies Unicode NFC and consistent line endings while retaining a mapping between raw and normalized positions. It creates stable blocks and identifiers, proposes conservative language hints, and normalizes recognized placeholders without discarding original evidence. Each extraction record retains raw text, normalized text, their hashes, counts, method, status, warnings, and ordered blocks.

5.5 Stage 4 — Labeling, assembly, and validation

Model-assisted labeling is coordinate-only. The local model can identify document kind, jurisdictions, sections, parties, and placeholders by referencing existing source spans. It is prohibited from generating replacement legal text. Calls are sequential, temperature is zero, responses must satisfy a strict schema, coordinates must resolve to source text, and retry is limited to malformed structured output.

Assembly uses validated coordinates to create the provider-neutral canonical record. Any meaningful content that cannot be classified remains under `unparsed_source_content`. Validation checks schemas, hashes, coordinates, placeholders, cross-references, order, eligibility, source evidence, and provider-neutrality.

5.6 Stage 5 — Fixed pilot

The pilot uses the exact paths and reasons in `config/pilot-manifest.json`; it does not randomly sample the corpus.

```
python src\pilot.py
```

To select an installed local model tag and context explicitly:

```
python src\pilot.py --model qwen3.5:4b-q4_K_M --num-ctx 4096
```

The configured model is never downloaded automatically. If it is unavailable, the pipeline records the blocker and does not fabricate labels.

5.7 Stage 6 — Full corpus

The current deterministic full-corpus route can be reproduced without model assistance:

```
python src\full_corpus.py --skip-review --no-model
```

The model-assisted command documented for the locally available Qwen tag is:

```
python src\full_corpus.py --model qwen2.5:3b --num-ctx 4096 --skip-review
```

Processing is sequential and resumable. Compatible completed groups are reused automatically. `--force` regenerates every group, `--doc-timeout 120` changes the isolated legacy-DOC timeout, and `--allow-count-drift` explicitly relaxes the recorded corpus-count check. Exact duplicates are processed once while every physical path remains in the manifest.

6. Canonical Contract Templates and Review

6.1 Canonical record purpose

The canonical record is the richest structured representation of an extracted contract-like document. It is designed for audit and review rather than minimum runtime size. Its required top-level fields are:

```
{
  "schema_version": "...",
  "template_id": "...",
  "source_files": [],
  "document": {},
  "jurisdictions": [],
  "parties": [],
  "variables": [],
  "sections": [],
  "unparsed_source_content": [],
  "quality": {},
  "sanitization_history": [],
  "audit_history": [],
  "model_assistance": {},
  "versions": {}
}
```

This representation preserves identity, source tracking, document classification, jurisdiction, language, parties, variables, sections, clauses, unparsed evidence, quality status, model-assistance provenance, and review history. It intentionally contains no fabricated embeddings and no vendor publication settings.

6.2 Classification and template state

The pipeline distinguishes document kind from template state. A file may be classified as a contract template, completed contract, form, policy, letter, resolution, court filing, or unknown. A contract-like record may separately have a blank, completed, mixed, or unknown template state. This avoids treating every file under a folder called `templates` as a safe blank contract template.

Completed or mixed documents may contain personal information or negotiated facts. They cannot be directly promoted to a reusable contract template. Reclassification requires recorded sanitization, privacy approval, structure approval, legal approval, a named human actor, a review-record identifier, and a reason. Initial classification, source identity, sanitization history, reclassification history, and audit actions remain attached.

6.3 Review workspace

The local review dashboard is started with:

```
python review_app.py
```

It binds only to `127.0.0.1:5000`. A reviewer supplies a name and works with resumable records stored under `review-workspace/records/<review_id>/`. The application creates immutable references to generated artifacts, maintains atomic working-copy revisions, validates corrections, and constrains sanitized text to source evidence. It does not overwrite source documents or generated artifact records.

6.4 Review dimensions

Depending on classification and content, records can require:

1. Human classification review.

1. Structure review for section and clause organization.
1. Legal review for documentary and jurisdictional suitability.
1. Privacy review and sanitization for completed or mixed records.
1. Confirmation of placeholder and variable mappings.
1. Final disposition and training-eligibility decision bound to the exact artifact hash.

The review state is part of the data contract, not an informal checklist. Production runtime export and production Alpaca export both enforce it.

6.5 Eligibility safeguard

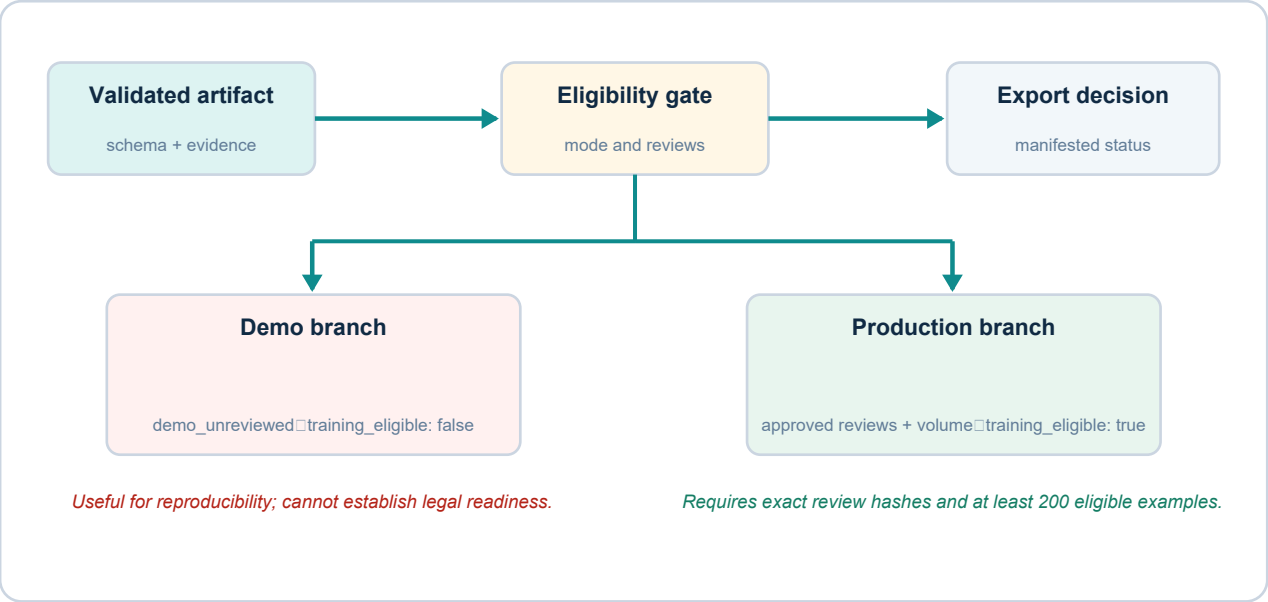


Figure 3. Separation between demonstrator outputs and production eligibility.

Demonstration mode exists to show the complete engineering path without weakening production gates. Records generated with `--demo-unreviewed` retain the status `demo_unreviewed`. Their manifests record `training_eligible: false`, even if their schemas, evidence, and target reconstruction are valid. Production mode requires approved source eligibility and review information bound to the target hash.

7. Qwen Enrichment and Compact Runtime Records

7.1 Why enrichment is separate

The canonical JSON remains the evidence-bearing source of truth. Qwen enrichment is a separate post-processing flow that receives only compact section or clause packets and existing variable metadata. It does not receive permission to rewrite source text, create law, or overwrite canonical artifacts.

Only `contract_template` and `completed_contract` records generate packets. The response schema requires packet identity, template identity, section identity and kind, section requirement status, clause labels, variable labels, and warnings. Merging occurs only when response identity, schema, and grounding checks pass.

7.2 Enrichment commands

Create compact packets from full-corpus canonical records:

```
python src\qwen_enrichment.py packets --canonical-dir artifacts\full\records --output-dir artifacts\qwen-
packets
```

Run the local Qwen model over packets:

```
python src\qwen_enrichment.py run --packets-dir artifacts\qwen-packets --model qwen2.5:3b --num-ctx 4096
```

Retry only recorded failures when required:

```
python src\qwen_enrichment.py run --packets-dir artifacts\qwen-packets --model qwen2.5:3b --num-ctx 4096 --
retry-failures
```

Merge schema-valid labels into a separate canonical tree:

```
python src\qwen_enrichment.py merge --canonical-dir artifacts\full\records --packets-dir artifacts\qwen-
packets --output-dir artifacts\enriched-canonical
```

The current merge summary records 90 merged contract records, 570 failed contract-record merges, and 613 skipped non-contract records. Failure remains visible instead of being converted into guessed metadata.

7.3 Compact runtime projection

The runtime record reduces a canonical record to the information required for contract retrieval and downstream data preparation. Required fields include runtime status, template identity, canonical reference, document metadata, jurisdictions, parties, variables, sections, and unparsed content.

Production projection accepts only reviewed and eligible blank contract templates. The command is:

```
python src\runtime_templates.py --canonical-dir artifacts\enriched-canonical\records --output-dir artifacts\
runtime
```

The current production-gated summary generated zero records and rejected 29 considered records, demonstrating that the eligibility boundary is active. For the explicitly unreviewed engineering demonstration, the command is:

```
python src\runtime_templates.py --canonical-dir artifacts\enriched-canonical\records --output-dir artifacts\
runtime-demo --demo-unreviewed
```

This produced 90 compact records with `runtime_status: demo_unreviewed` and no runtime-projection rejections.

7.4 Runtime record role

The runtime record is neither an Alpaca row nor a replacement for the canonical audit record. It provides a compact representation that can be retrieved, matched to Agent 1 facts, and converted into an evidence package. Its `canonical_ref` makes it possible to return to the richer source and review history whenever a downstream question arises.

8. Agent 1 Demonstration Inputs

8.1 Fixed interface

The Agent 1 interface is supplied in `features/contract-template-dataset/agent_1_json_output.md` and implemented as `agent1-raw-payload-v1.schema.json`. Its required top-level fields remain unchanged:

```
{
  "metadata": {
    "session_id": "...",
    "user_id": "...",
    "date_generation": "...",
    "pays": "...",
    "juridiction": "...",
    "type_document": "contrat"
  },
  "type_contrat": "...",
  "sous_type": "...",
  "parties": [],
  "informations_principales": {},
  "informations_generales": {
    "date_creation_contrat": "...",
    "langue_contrat": "..."
  },
  "informations_manquantes": [],
  "autres_informations": {}
}
```

Agent 2 may validate and normalize this payload, but it may not silently alter the fixed raw interface. The raw record is retained for audit, while a separate sanitized projection is used for the model and Alpaca export.

8.2 Generation modes

The demonstration generator uses the compact runtime record to create one possible Agent 1 payload per contract:

1. Blank templates receive visibly fictional facts suitable for exercising variable binding.
1. Completed contracts use only facts already present in source-extracted runtime evidence.
1. Each extracted fact cites a runtime clause or unparsed-content identifier.
1. Required values that cannot be established become the literal `AMBIGUITY` and produce validation warnings.
1. Completed-contract records retain `privacy_review_required: true`.

Qwen proposes document classifications, parties, variable bindings, and additional facts. Deterministic code validates the proposal and assembles the unchanged Agent 1 payload. Qwen cannot add clauses, laws, citations, or approval status.

8.3 Command

```
python src\agent1_demo_inputs.py --runtime-dir artifacts\runtime-demo --model qwen2.5:3b --num-ctx 32768 --
num-predict 8192 --case-date 2026-08-14
```

The operation is sequential and resumable. `--timeout-seconds` changes the per-call boundary, `--base-url` changes the local Ollama endpoint, and `--overwrite-proposals` explicitly replaces previously generated proposals. For each runtime directory it can retain:

1. `agent1-qwen-response.json`: raw structured model response.
1. `agent1-qwen-proposal.json`: normalized proposal.
1. `agent1-input.json`: assembled fixed-interface payload when valid.
1. `agent1-input-validation.json`: errors, warnings, evidence, and eligibility state.

8.4 Measured outcome

The current Agent 1 summary audits 90 runtime records: 53 blank templates and 37 completed contracts. It records 65 generated valid inputs, 17 blocked records, and eight Qwen failures. The generated inputs remain `demo_unreviewed` and are not automatically promoted by their successful schema validation.

8.5 Sanitization before Agent 2

`session_id` and `user_id` are application identifiers, not model features. The Agent 2 intake adapter removes them from the model-facing facts and all Alpaca artifacts. It records forbidden-identifier paths, raw and sanitized hashes, counts, normalization audit entries, errors, warnings, and a reference back to the raw input. This allows privacy-oriented minimization without destroying application traceability.

9. Agent 2 Grounding and Target Reconstruction

9.1 Generation-facts record

The sanitized `agent2-generation-facts-v1` record contains:

Field	Purpose
`schema_version`	Versioned interface identity
`record_status`	Demonstration or eligible state
`template_id`	Link to runtime and canonical artifacts
`document`	Contract type, language, and jurisdiction facts
`parties`	Normalized party facts supplied through Agent 1
`variable_bindings`	Explicit mapping from facts to template variables
`unmapped_facts`	Supplied information that cannot be bound deterministically
`warnings`	Ambiguity and normalization audit messages

Duplicate or conflicting facts are not resolved by an undocumented precedence rule. They are rejected or retained as warnings according to the implemented validation behavior.

9.2 Retrieved evidence package

The `retrieved-contract-evidence-v1` record contains a template object and the exact clauses supplied to Agent 2. This artifact represents the documentary retrieval boundary in the demonstrator. It records template identity, document metadata, source identities, canonical references, clauses, and retrieval status.

The evidence package is generated before the target is built. This ordering matters: the target is not accepted and then retroactively “supported” by a citation. Instead, the retrieved record defines the documentary material available for construction.

9.3 Target reconstruction

Compact runtime records do not always carry one complete linear sequence across sections and unparsed content. The implemented deterministic path therefore uses canonical and extraction source order to reconstruct the target. It preserves complete ordered text, normalizes only the documented representation, and calculates target hashes.

Each passage in `contract-target-validation-v1` records its grounding and validation status. Checks cover source order, source-text coverage, hashes, template identity, passage evidence, required content available in the source, and the distinction between demonstration validity and production eligibility.

9.4 Contract-quality checks

At minimum, the source requires validation of:

1. Required sections.
1. Missing clauses.
1. Malformed or unreplaced placeholders.

1. Internal contradictions.
1. Legal hallucinations.
1. Legal drafting style.
1. MENA adaptation.
1. Lawyer review time and generation time.

The repository automatically enforces structural and provenance checks it can measure from artifacts. Human or model-based legal-quality KPIs require their source-defined measurement methods and approved evaluation rubrics. Their absence is never converted into a passing score.

> **Grounding rule.** A target may be valid for a reproducible demonstration because its passages resolve to source evidence while still remaining ineligible for production training because review and volume gates are separate concerns.

10. Alpaca Dataset Export

10.1 Standard row

The source defines one standard training shape and does not authorize an alternative production schema:

```
{
  "instruction": "What Agent 2 must do",
  "input": "Sanitized contract facts plus retrieved documentary evidence",
  "output": "Grounded target contract or reasoning result"
}
```

The three fields are deliberately distinct. `instruction` defines the task. `input` serializes the allowed facts and retrieval material. `output` contains the grounded target. Audit paths, eligibility, source hashes, and review details do not leak into the model-facing row; they remain in the paired manifest.

10.2 Export command

```
python src\alpaca_export.py --runtime-dir artifacts\runtime-demo --output-dir artifacts\alpaca-demo --
canonical-dir artifacts\enriched-canonical\records --extraction-dir artifacts\full\records --mode demo --
expected-records 90
```

The exporter audits all indexed runtime records. If an Agent 1 input is unavailable or invalid, that record is written to `rejected.jsonl`. A later rerun can process it after its upstream artifact appears; no force flag is required.

10.3 Per-record outputs

Valid records receive the following audit files beside the runtime artifact:

1. `agent2-generation-facts.json` — sanitized facts with forbidden identifiers removed.
1. `agent2-generation-validation.json` — mappings, source hashes, validation results, and normalization audit.
1. `retrieved-evidence.json` — exact documentary template and clause evidence.
1. `contract-target.txt` — target reconstructed from ordered extraction content.
1. `contract-target-validation.json` — passage-level grounding and quality checks.

Dataset-level outputs under `artifacts/alpaca-demo/` are:

File	Function
`alpaca.jsonl`	Strict three-field model-facing rows
`manifest.jsonl`	Row identity, source/template/clause IDs, paths, hashes, reviews, and eligibility

File	Function
`rejected.jsonl`	Audited runtime records that did not produce a row
`summary.json`	Counts, status, distributions, audit findings, and minimum-volume state

10.4 Current export result

The export audited 90 runtime records. It accepted 65 valid Agent 1 records, exported 65 Alpaca rows, and recorded 25 rejected records. The row distribution is:

Dimension	Count
Completed-contract sources	27
Contract-template sources	38
English	36
Arabic	26
French	3
Contract family `unknown`	65

The absence of resolved contract families prevents evidence-based claims about specialized adapter coverage. The summary reports zero eligible examples in demo mode and `minimum_volume_met: false` against the source-defined minimum of 200 eligible examples.

10.5 Production mode

The same exporter supports `--mode production`, but production mode requires approved upstream eligibility and exact review bindings. The demo flag is not a general bypass: schema, integrity, forbidden-identifier, evidence, and target-grounding checks remain mandatory.

11. Fine-Tuning Implementation and Handoff

11.1 Scope inside Agent 2

`src/finetune_slm.py` consumes an already exported Alpaca directory and prepares or trains only an Agent 2 adapter. Agent 1 remains the information collector. Agent 3 is not part of training. Retrieved documentary content remains the only legal source.

The entry point keeps heavy machine-learning imports lazy. Dataset audits and automated tests can therefore run on a workstation without loading model weights. Model training is guarded by a CUDA and VRAM check.

11.2 Implemented QLoRA profile

Parameter	Implemented default
Base model	`Qwen/Qwen2.5-3B-Instruct`
Adaptation	4-bit QLoRA
Quantization	NF4 with double quantization
LoRA rank	16
LoRA alpha	32
LoRA dropout	0.05
Epochs	2
Learning rate	`1e-4`
Scheduler	Linear, zero warmup steps
Optimizer	`paged_adamw_8bit`
Maximum sequence length	8192 tokens
Train/evaluation batch size	1 per device
Gradient accumulation	4
Seed and data seed	42
Minimum GPU memory gate	16 GB
Split	Deterministic grouped 80/10/10
Loss	Assistant completion tokens only
Truncation	None; over-limit rows are excluded unchanged

The QLoRA configuration loads the base in four bits, prepares it for k-bit training with gradient checkpointing, and makes only LoRA parameters trainable. LoRA targets cover the configured projection modules. The trainer evaluates and saves each epoch, retains at most two checkpoints, reloads the best model by evaluation loss, and writes separate train, validation, and test diagnostic metrics.

11.3 Grouped split and leakage control

Rows related to the same source, template, or clause identity are grouped before assignment. This reduces the risk that near-identical evidence appears in both training and evaluation. Assignment is deterministic for seed 42 and seeks the 80/10/10 global distribution while retaining non-empty train, validation, and test sets.

Tokenization uses Qwen's chat template. Prompt tokens receive label `-100`, so only assistant completion tokens contribute to loss. Rows longer than 8192 tokens are not silently truncated; they are copied unchanged into `excluded.jsonl` with their token count and reason.

11.4 Dataset audit before model loading

Before any model weight is loaded, the script validates:

1. Presence and integrity of `alpaca.jsonl`, `manifest.jsonl`, and `summary.json`.
1. One-to-one row and manifest relationships.
1. Row, input, output, source, template, and clause hashes.
1. Status and training-eligibility metadata appropriate to the selected mode.
1. Absence of forbidden identifiers in model-facing content.

1. Minimum eligible volume for production mode.
1. Safe separation between source dataset paths and output paths.

This preflight is an important operational result because it prevents a malformed or mislabeled dataset from consuming GPU time.

11.5 Source profile versus implemented profile

The R&D source recommends Llama 3.1 8B or Mistral 7B, with GPT-4o-mini through Azure as an alternative. It recommends LoRA and presents QLoRA as an alternative. It gives a maximum sequence length of 4096, learning rates of `2e-5` for Llama and `5e-6` for Mistral, rank 16, alpha 32, two epochs, and GPU-dependent batches of four or eight.

The repository's executable demonstration profile uses Qwen2.5 3B, QLoRA, 8192 tokens, `1e-4`, and micro-batch one with gradient accumulation four. These values are implementation defaults for this guarded entry point; they do not silently resolve the source's final production model, provider, method, or infrastructure decisions.

> **AMBIGUITY: Base model and adaptation profile.**
> Source A: The project source recommends Llama 3.1 8B or Mistral 7B, prefers LoRA, and lists QLoRA as an alternative.
> Source B: The implemented executable profile uses Qwen2.5 3B with QLoRA for the current handoff.
> Decision required: Select and approve the production base model, adaptation variant, sequence length, learning rate, GPU, and provider before production training and evaluation.

12. Fine-Tuning and Packaging Commands

12.1 Install the isolated stack

A CUDA-enabled PyTorch build compatible with the selected training host must be installed first. The repository then supplies pinned adaptation dependencies:

```
python -m pip install -r requirements-finetune.txt
```

The stack includes Transformers, TRL, PEFT, Accelerate, Datasets, bitsandbytes, Safetensors, and JSON Schema validation. PyTorch is intentionally not pinned because its wheel must match CUDA.

12.2 Exact-token preflight

```
python src\finetune_slm.py artifacts\alpaca-demo --output-dir artifacts\finetuned\hannibal-demo-preflight --
allow-unreviewed-demo --preflight-only
```

Preflight uses the real tokenizer and chat template but does not load model weights. It writes source hashes, token counts, length exclusions, deterministic split files, split manifests, a dataset report, and `status.json` with `preflight_complete` and `production_ready: false`.

12.3 Execute the guarded demonstration profile

```
python src\finetune_slm.py artifacts\alpaca-demo --output-dir artifacts\finetuned\hannibal-demo-qwen2.5-3b --
allow-unreviewed-demo
```

The `--allow-unreviewed-demo` option is mandatory for the current demonstration export. It does not bypass schema, hash, source-grounding, identifier, or minimum-volume reporting controls. A production dataset must satisfy production status and eligibility without this option.

12.4 Resume from a checkpoint

```
python src\finetune_slm.py artifacts\alpaca-demo --output-dir artifacts\finetuned\hannibal-demo-qwen2.5-3b --
allow-unreviewed-demo --resume-from-checkpoint artifacts\finetuned\hannibal-demo-qwen2.5-3b\checkpoints\
checkpoint-<step>
```

The checkpoint path must belong to the same deliberate output scope. Dataset validation and preparation still occur before training resumes.

12.5 Override profile values explicitly

```
python src\finetune_slm.py artifacts\alpaca-demo `
  --output-dir artifacts\finetuned\hannibal-demo-qwen2.5-3b `
  --allow-unreviewed-demo `
  --max-sequence-length 8192 `
  --epochs 2 `
  --learning-rate 1e-4 `
  --lora-rank 16 `
  --lora-alpha 32 `
  --lora-dropout 0.05 `
  --per-device-train-batch-size 1 `
  --per-device-eval-batch-size 1 `
  --gradient-accumulation-steps 4 `
  --seed 42 `
  --minimum-vram-gb 16
```

12.6 Merge and register with Ollama

```
python src\finetune_slm.py artifacts\alpaca-demo --output-dir artifacts\finetuned\hannibal-demo-qwen2.5-3b --
allow-unreviewed-demo --export-ollama --llama-cpp-dir C:\path\to\llama.cpp --ollama-model hannibal-qwen2.5-3b-
demo
```

This command trains the adapter, reloads the same Hugging Face base model, merges the PEFT adapter, saves a merged model, runs `convert_hf_to_gguf.py`, creates an F16 GGUF, quantizes it to `Q4_K_M`, writes a Modelfile with the selected context length, registers the model through `ollama create`, and verifies it through `ollama show`. It does not attach LoRA Safetensors directly to an unrelated installed Ollama base.

The packaging path requires an existing llama.cpp checkout containing `convert_hf_to_gguf.py`, a built `llama-quantize` executable, and `ollama` on `PATH`.

13. Verified Results

13.1 Pilot results

The fixed pilot summary records:

Measure	Result
Full physical inventory	1,276
PDF / DOC / DOCX	955 / 297 / 24
Exact duplicate groups	1
Selected pilot files	36
Unique pilot content groups	35
Successful pilot extractions	34
Encrypted pilot groups	1
Canonical proposals	34
Canonical validation failures	0
Review queue records	35

Measure	Result
Source files changed	0

The requested `qwen3.5:4b-q4_K_M` tag was unavailable during that pilot. The summary records zero model-assisted pilot records rather than substituting the installed `qwen2.5:3b` baseline without an explicit decision.

13.2 Full-corpus results

Measure	Result
Physical source files	1,276
Unique content groups	1,275
Successful extraction groups	1,273
Encrypted groups	2
Processing failures	0
Contract-like demonstration candidates	660
Excluded groups	615
Blank templates	401
Completed contracts	259
Source files changed	0

Document-kind routing recorded 401 contract templates, 259 completed contracts, 14 court filings, 97 forms, 39 letters, 33 policies, 15 resolutions, 415 unknown records, and two records without a document kind. These are deterministic routing proposals, not lawyer-approved legal classifications.

13.3 Downstream results

Stage	Generated/accepted	Rejected/failed/skipped	Status
Qwen enrichment merge	90	570 failed; 613 non-contract skipped	Separate enriched artifacts
Production-gated runtime projection	0	29 rejected	Eligibility gate active
Demonstration runtime projection	90	0	`demo_unreviewed`
Agent 1 generation	65	17 blocked; 8 Qwen failures	`demo_unreviewed`
Alpaca export	65	25 rejected	`training_eligible: false`

13.4 Automated test baseline

On 17 August 2026, the complete suite produced:

```
.................................................                    [100%]
51 passed in 55.40s
```

The tests cover extraction, inventory, normalization, validation, full-corpus behavior, review workspace and application routes, Qwen enrichment, Agent 1 input generation, grounded Agent 2 preparation, Alpaca export, and fine-tuning safeguards.

13.5 Interpretation

The measured results establish that the repository can account for the full source corpus, preserve its integrity, create strict intermediate records, expose failures, generate compact runtime inputs, and build grounded Alpaca data under a demonstrator status. They do not establish the source-defined legal-quality KPIs. Training loss and perplexity are also explicitly treated as diagnostics rather than evidence of legal correctness.

14. Quality, Safety, and Audit Controls

14.1 Source integrity

Inventory hashes every source before processing. Pilot and full-corpus orchestration repeat the hashes after processing and fail if a path changed. Both recorded summaries report `all_templates_unchanged: true` and an empty changed-path list.

14.2 Grounding and hallucination controls

The pipeline reduces unsupported content through multiple independent controls:

1. Qwen labels are restricted to existing source coordinates.
1. Canonical assembly is deterministic and preserves unparsed evidence.
1. Runtime records point back to canonical records.
1. Retrieved-evidence packages are materialized before target construction.
1. Target passages are checked against source extraction.
1. Alpaca manifests preserve template, source, and clause identities and hashes.
1. Fine-tuning audits all row/manifest relationships before model loading.

These controls support the required zero-legal-hallucination objective, but the KPI itself requires the source-defined measurement process and cannot be declared passed from structural validation alone.

14.3 Privacy controls

Completed contracts receive stronger treatment because they can contain negotiated values or personal data. Privacy review and sanitization are mandatory before production template reclassification. Agent 1's raw session and user identifiers remain in audit-only files and are prohibited from model-facing facts and Alpaca rows.

14.4 Placeholder and completeness controls

The canonical feature uses `[UPPER_SNAKE_CASE]` rendered placeholders paired with `lower_snake_case` variable identifiers. Validators check malformed tokens, duplicate or unresolved mappings, required sections, clause references, and ordered section content. Missing information is represented explicitly rather than filled with plausible values.

14.5 Status and eligibility controls

Status is enforced at every boundary. A valid schema does not imply human review. Human review does not automatically imply privacy approval. A compact runtime record is not an Alpaca row. A demonstration row is not a production row. Production eligibility requires the correct combination of source status, exact reviews, target hash, and minimum volume.

14.6 KPI discipline

KPI	Source target	Required evidence
Required sections	100% or greater than 99%	Compare with approved contract plan
Malformed placeholders	0	Automatic verification
Correctly replaced variables	Greater than 99%	Placeholder replacement check
Missing clauses	Less than 2%	Compare with Legal Planner
Internal contradictions	Less than 1%	Defined automated validation
Legal hallucinations	0%	Tracked reports and evidence audit
Drafting/legal style	Greater than 85%	Defined GPT-4o evaluation rubric
MENA adaptation	Greater than 80%	Local-lawyer evaluation
Accepted without major change	Greater than 90%	Lawyer feedback
Lawyer correction time	Less than 10 minutes	Measured review time
Generation time	Less than 15 seconds	Application logs

No KPI is claimed as passed in this report because the conflicting threshold and several measurement rubrics remain unresolved.

15. Critical Ambiguities and Decision Gates

15.1 Architecture and dataset ownership

AMBIGUITY: Dataset D ownership.
Source A: The system architecture defines exactly three agents.
Source B: Dataset D says contract-to-omissions/inconsistencies examples are generated by “Agent 4.”
Decision required: Identify the intended existing component or formally revise the architecture before Dataset D generation is assigned.

15.2 Required-section KPI

AMBIGUITY: Required-section completeness threshold.
Source A: The objective table requires 100% of mandatory sections.
Source B: The KPI table requires greater than 99%.
Decision required: Select one threshold and its measurement method before reporting this KPI.

15.3 Schedule

AMBIGUITY: Integration and deployment calendar.
Source A: Detailed Phase 4 assigns integration and production A/B deployment to weeks 7–8.
Source B: The global calendar assigns integration, A/B deployment, KPI measurement, and adjustments to weeks 9–12.
Decision required: Select the controlling schedule and release authority.

15.4 Runtime payloads

AMBIGUITY: Production Agent 2 interfaces.
Source A: Agent 1 has a fixed supplied JSON interface and the architecture requires structured handoffs.
Source B: The source PDFs do not define exact Agent 1-to-Agent 2 or Agent 2-to-Agent 3 production schemas.
Decision required: Approve or revise the repository's versioned Agent 2 projection before treating it as a production interface.

15.5 Adapter routing

AMBIGUITY: NDA adapter routing.
Source A: Commercial includes services, NDA, MSA, and partnership.
Source B: NDA is also defined as its own adapter family with confidentiality and NDA coverage.
Decision required: Define routing precedence, overlap handling, and evaluation separation.

15.6 Evaluation

AMBIGUITY: Evaluation rubrics and statistical acceptance.
Source A: The project defines targets for drafting style, MENA adaptation, lawyer acceptance, correction time, generation time, and 50 test contracts with a 10% A/B exposure.
Source B: The exact GPT-4o prompt, legal-style rubric, MENA rubric, baseline, split, and statistical rule are missing.
Decision required: Approve reproducible rubrics, datasets, and acceptance calculations before KPI conclusions.

15.7 Non-contract destination

AMBIGUITY: Non-contract corpus destination.
Source A: Every collected file must remain inventoried and available for future processing.
Source B: The current feature is limited to contract templates, while the corpus includes policies, forms, letters, filings, resolutions, and unknown records.
Decision required: Define separate destinations, a separately reviewed transformation, or audit-only retention.

16. Deployment-Readiness Gates and Recommendations

16.1 Data readiness

Before a production adapter dataset can be declared ready, the project must complete the following evidence-backed gates:

1. Review and sanitize the selected source records.
2. Bind structure, legal, and privacy decisions to exact record hashes.
3. Resolve contract families and adapter routing.
4. Expand to at least 200 eligible examples, with 500–1,000 preferred by the source.
5. Preserve source, jurisdiction, language, and clause identities in manifests.
6. Re-export in production mode and confirm `minimum_volume_met: true`.
7. Run exact-token preflight and inspect every over-limit exclusion.

16.2 Infrastructure readiness

The repository's implemented profile requires a CUDA-enabled NVIDIA GPU and defaults to a minimum 16 GB VRAM gate. The source separately lists A10G 24 GB as minimal, A100 40 GB as recommended, and H100 80 GB as optimal, with Azure NCas-v4, RunPod, Vast.ai, and Google Colab Pro+ as candidates. These are source options rather than a selected budget or provider.

The training host must also have a matching CUDA PyTorch build, the isolated fine-tuning dependencies, sufficient storage for base and merged weights, and a deliberate output directory separate from the dataset.

16.3 Evaluation readiness

Evaluation should preserve three layers:

1. **Technical diagnostics:** training/evaluation loss, perplexity, token lengths, exclusions, hashes, and split leakage checks.
1. **Contract-quality checks:** mandatory sections, clauses, placeholders, contradictions, structure, and generation latency.
1. **Legal/human checks:** documentary grounding, MENA adaptation, lawyer acceptance, and measured correction time.

Loss or perplexity must not be used as a substitute for legal review. The source calls for validation on 50 test contracts and an A/B production exposure of 10%, but these activities require approved rubrics, release controls, and reviewer participation.

16.4 Runtime integration readiness

The final adapter must be loaded inside Agent 2. Integration must preserve the sequence `analyze → choose indexes → retrieve → select clauses → build → validate → risk analysis`. Adapter selection must not bypass RAG, and fallback behavior must not invite unsupported legal completion from the base model.

16.5 Recommended execution order

1. Resolve the base-model and adapter-family decisions.
2. Complete legal, structure, and privacy review for an initial focused contract family.
3. Produce a production-mode Alpaca export meeting the minimum eligible volume.
4. Run token preflight and inspect prepared split manifests.
5. Execute the selected LoRA/QLoRA profile on approved GPU infrastructure.
6. Evaluate technical diagnostics and the approved legal-quality test set.
7. Package the adapter or merged GGUF and verify local inference.
8. Integrate dynamic adapter loading into Agent 2 while retaining RAG-first behavior.
9. Conduct lawyer validation and authorized A/B measurement.
10. Record every result using the specified measurement method.

17. Conclusion

The repository converts an initially heterogeneous legal-document collection into a controlled series of source-grounded artifacts. Its principal achievement is not merely file conversion. It creates an auditable chain between physical legal documents, extracted evidence, canonical templates, runtime records, Agent 1 facts, Agent 2 retrieval packages, grounded targets, and strict Alpaca rows.

The implementation preserves the most important architectural boundary in Hannibal V2: legal content originates in documentary retrieval, while model adaptation improves how Agent 2 reasons, organizes, and drafts from that material. Agent 1 remains a collector and Agent 3 remains a formatter. Demonstration and production eligibility remain separate, and failures or ambiguities are recorded rather than hidden.

Measured artifacts show complete corpus accounting, 1,273 successful unique-group extractions, 660 contract-like candidates, 90 runtime records, 65 grounded Alpaca rows, unchanged source hashes, and 51 passing tests. The guarded QLoRA handoff adds dataset audits, exact tokenization, leakage-aware splits, completion-only supervision, adapter isolation, diagnostics, and an Ollama packaging path.

The remaining work is governed by explicit decisions and evidence gates: human review, privacy treatment, eligible dataset volume, production model selection, adapter routing, evaluation rubrics, GPU execution, legal-quality measurement, and Agent 2 integration. Because these gates are visible and enforceable, the project can advance without confusing technical plausibility with legal correctness.

Appendix A. Complete Command Reference

A.1 Base environment and tests

```
python -m pip install -r requirements.txt
python -m pytest -q
```

A.2 Inventory and extraction

```
python src\inventory.py --templates-dir templates --output artifacts\manifest.jsonl --strict-counts
python src\extract.py <source-path> --output <extraction.json>
python src\extract.py <source-path> --output <extraction.json> --manifest-record <manifest-record.json>
```

A.3 Pilot and corpus

```
python src\pilot.py
python src\pilot.py --model qwen3.5:4b-q4_K_M --num-ctx 4096
python src\full_corpus.py --skip-review --no-model
python src\full_corpus.py --model qwen2.5:3b --num-ctx 4096 --skip-review
python src\full_corpus.py --model qwen2.5:3b --num-ctx 4096 --skip-review --doc-timeout 120
```

A.4 Review and enrichment

```
python review_app.py
python src\qwen_enrichment.py packets --canonical-dir artifacts\full\records --output-dir artifacts\qwen-
packets
python src\qwen_enrichment.py run --packets-dir artifacts\qwen-packets --model qwen2.5:3b --num-ctx 4096
python src\qwen_enrichment.py run --packets-dir artifacts\qwen-packets --model qwen2.5:3b --num-ctx 4096 --
retry-failures
python src\qwen_enrichment.py merge --canonical-dir artifacts\full\records --packets-dir artifacts\qwen-
packets --output-dir artifacts\enriched-canonical
```

A.5 Runtime, Agent 1, and Alpaca

```
python src\runtime_templates.py --canonical-dir artifacts\enriched-canonical\records --output-dir artifacts\
runtime
python src\runtime_templates.py --canonical-dir artifacts\enriched-canonical\records --output-dir artifacts\
runtime-demo --demo-unreviewed
python src\agent1_demo_inputs.py --runtime-dir artifacts\runtime-demo --model qwen2.5:3b --num-ctx 32768 --
num-predict 8192 --case-date 2026-08-14
python src\alpaca_export.py --runtime-dir artifacts\runtime-demo --output-dir artifacts\alpaca-demo --
canonical-dir artifacts\enriched-canonical\records --extraction-dir artifacts\full\records --mode demo --
expected-records 90
```

A.6 Fine-tuning handoff and packaging

```
python -m pip install -r requirements-finetune.txt
python src\finetune_slm.py artifacts\alpaca-demo --output-dir artifacts\finetuned\hannibal-demo-preflight --
allow-unreviewed-demo --preflight-only
python src\finetune_slm.py artifacts\alpaca-demo --output-dir artifacts\finetuned\hannibal-demo-qwen2.5-3b --
allow-unreviewed-demo
python src\finetune_slm.py artifacts\alpaca-demo --output-dir artifacts\finetuned\hannibal-demo-qwen2.5-3b --
allow-unreviewed-demo --resume-from-checkpoint <checkpoint-path>
python src\finetune_slm.py artifacts\alpaca-demo --output-dir artifacts\finetuned\hannibal-demo-qwen2.5-3b --
allow-unreviewed-demo --export-ollama --llama-cpp-dir C:\path\to\llama.cpp --ollama-model hannibal-qwen2.5-3b-
demo
```

A.7 Rebuild this report

```
python reports\generate_report_pdf.py
python reports\generate_report_pdf.py --source reports\Hannibal_V2_Technical_Report.md --output reports\
Hannibal_V2_Technical_Report.pdf
```

Appendix B. Schema and Artifact Matrix

Schema/artifact	Producer	Consumer	Audit role
`manifest-record-v1`	Inventory	Extraction and orchestration	Physical identity and hash
`extraction-record-v1`	Extractor/normalizer	Assembly and target builder	Ordered source text and mappings
`labeling-response-v1`	Local model boundary	Canonical assembler	Coordinate-only proposals
`contract-template-v1`	Assembler	Review and enrichment	Provider-neutral evidence record
`review-record-v1`	Pilot/full corpus	Review UI	Required review decisions
`review-workspace-record-v1`	Review application	Eligibility checks	Named revisions and approvals
`qwen-contract-label-packet-v1`	Enrichment packetizer	Local Qwen	Compact source text scope
`qwen-contract-label-response-v1`	Local Qwen	Enrichment merge	Schema-bound labels
`contract-template-runtime-v1`	Runtime projection	Agent 1 and Agent 2 preparation	Compact retrieval record
`agent1-raw-payload-v1`	Agent 1 generator	Agent 2 intake	Fixed raw information interface
`agent2-generation-facts-v1`	Agent 2 intake	Alpaca serializer	Sanitized model-facing facts
`agent2-generation-validation-v1`	Agent 2 intake	Audit/export gate	Hashes, mappings, identifiers
`retrieved-contract-evidence-v1`	Target builder	Agent 2 input	Exact template/clause evidence
`contract-target-validation-v1`	Target builder	Alpaca export	Passage grounding and target hash
`alpaca-record-v1`	Alpaca exporter	Fine-tuning entry point	Three model-facing fields
`alpaca-export-manifest-v1`	Alpaca exporter	Fine-tuning audit	Sources, hashes, review, eligibility

Appendix C. Artifact Directory Guide

Path	Contents
`artifacts/manifest.jsonl`	One inventory record per physical source file
`artifacts/pilot/records/`	Pilot extraction, canonical, model, and validation artifacts
`artifacts/review-queue/`	Review records for unique pilot content groups
`artifacts/full/records/`	Resumable full-corpus artifacts by content-group key
`artifacts/full/demo-candidates.jsonl`	Contract-like, valid, unreviewed candidate index
`artifacts/full/excluded.jsonl`	Non-contract, failed, unreadable, invalid, or unknown routing
`artifacts/qwen-packets/`	Enrichment packets, responses, failures, and indexes
`artifacts/enriched-canonical/records/`	Separate canonical copies with accepted labels merged
`artifacts/runtime/`	Production-gated compact-runtime attempt
`artifacts/runtime-demo/records/`	Ninety demonstration runtime records and downstream audit files
`artifacts/alpaca-demo/alpaca.jsonl`	Sixty-five strict demonstration rows
`artifacts/alpaca-demo/manifest.jsonl`	Non-model row provenance and status
`artifacts/alpaca-demo/rejected.jsonl`	Twenty-five rejected runtime records
`review-workspace/records/`	Resumable human working copies and revision state

For one accepted runtime record, the audit chain can be followed from its runtime `canonical_ref`, through canonical source IDs, to extraction blocks and physical manifest hashes. Its Agent 1 validation points to the raw input. Its Agent 2 validation points to sanitized facts, retrieved evidence, and target validation. Its dataset manifest binds the exact row and output hash back to those identities.

Appendix D. Glossary

Term	Meaning in this project
Agent 1	Information Collector that gathers user facts and never drafts
Agent 2	Hannibal Core; owns legal analysis, RAG, construction, validation, and risk
Agent 3	Document Formatter with no legal logic
Alpaca row	Strict `instruction`, `input`, and `output` training record
Canonical record	Rich provider-neutral source and audit representation
Clause ID	Stable identifier used to trace retrieved or reconstructed clause evidence
Content group	One unique byte-content identity; may have multiple physical paths
Demo candidate	Valid contract-like artifact retained for engineering demonstration, not approval
Documentary index	RAG-accessible source of blueprint, contract, clause, or risk evidence
Exact duplicate	Multiple physical paths with the same SHA-256 content hash
Grounding	Demonstrable relationship between target content and documentary source evidence
LoRA	Parameter-efficient adaptation method preferred by the project source
QLoRA	LoRA training over a quantized base model; implemented in the current entry point
RAG	Retrieval-Augmented Generation; supplies the legal documentary content
Runtime record	Compact template representation used after canonical review/projection
Training eligibility	Explicit reviewed status required before production fine-tuning use
Unparsed content	Meaningful extracted source text retained when it cannot be safely classified

Appendix E. Source Traceability

Material claim	Primary repository source
Hannibal purpose and three-agent architecture	`Hannibal V2.pdf`, sections 1–6
Law comes exclusively from documentary indexes	`Hannibal V2.pdf`; `architecture.md`
Fine-tuning problems, objectives, volumes, strategy, infrastructure, roadmap, and KPIs	`Hannibal R&D.pdf`, sections 7–14; `context.md`
Blueprint, Q&A, risk, and clause example families	`contrat_example.md`
Fixed Agent 1 payload	`features/contract-template-dataset/agent_1_json_output.md`
Canonical dataset decisions and review safeguards	`features/contract-template-dataset/PLAN.md`
Grounded Agent 2 and Alpaca decisions	`features/agent2-grounded-finetuning/PLAN.md`
Pipeline modules and commands	`README.md`; executable `--help` interfaces under `src/`
Pilot measurements	`artifacts/pilot-run-summary.json`
Full-corpus measurements	`artifacts/full/run-summary.json`
Enrichment and runtime measurements	summaries under `artifacts/enriched-canonical/`, `artifacts/runtime/`, and `artifacts/runtime-demo/`
Alpaca counts and distributions	`artifacts/alpaca-demo/summary.json`
Fine-tuning defaults and safeguards	`src/finetune_slm.py`; `requirements-finetune.txt`
Automated test baseline	`tests/`; local `python -m pytest -q` run on 17 August 2026

E.1 Report integrity statement

This report does not reproduce or invent legal clauses. It does not interpret the displayed example vector fragments as complete embeddings. It does not select an unresolved production base model, provider, GPU, KPI threshold, or schedule. All current numeric results are copied from generated repository summaries or the documented local test run. Commands correspond to executable repository interfaces inspected on the report date.

E.2 Referenced repository documents

1. E-Tafakna, `Hannibal V2.pdf`, technical specification, version 2.0, sections 1–6.
2. E-Tafakna, `Hannibal R&D.pdf`, R&D specification, sections 7–14.
3. E-Tafakna, `contrat_example.md`, example documentary record families.
4. `architecture.md`, Hannibal V2 source-grounded architecture.
5. `context.md`, compact project context and ambiguity register.
6. `implementation_roadmap.md`, source-preserving implementation roadmap.
7. `README.md`, implemented pipeline, commands, and measured results.
8. Feature plans, JSON Schemas, executable modules, tests, and generated artifact summaries in this repository.